CONFIGMAP

What is a ConfigMap in Kubernetes?

A **ConfigMap** is a Kubernetes object used to store **non-sensitive configuration data** separately from application code.

Think of it like this:

👉 **Application = Employee**
👉 **ConfigMap = Instructions given to employee**

The employee (application) can work differently based on the instructions (configuration)

ConfigMap Explained with a Detailed Flipkart Real-Time Architecture Example

ConfigMap Explained with a Detailed Flipkart Real-Time Architecture Example

Let's take a scenario that actually happens in large e-commerce companies like Flipkart.

Scenario: Flipkart Product Service

When you open Flipkart and search for:

```
iPhone 17
```

The request goes through multiple microservices:

```
User
  |
  ▼
Ingress
  |
  ▼
Product Service
  |
  ├── MySQL Database
  |
  ├── Redis Cache
  |
  ├── Elasticsearch
  |
  └── Recommendation Service
```

The Product Service needs many configurations.

Example Configurations

```
Database Host
Database Port

Redis Host
Redis Port

Log Level

Feature Enable/Disable

Product Count Per Page

Elasticsearch URL

Timeout Values
```

Without ConfigMap

Developer hardcodes everything.

Java

```java
public class ProductApplication {

   String dbHost="mysql-prod.flipkart.internal";
   String dbPort="3306";

   String redisHost="redis-prod.flipkart.internal";

   String logLevel="INFO";

   int maxProducts=100;
}
```

Now imagine tomorrow:

Product team says:

```
Show only 50 products per page
```

What happens?

Developer must:

Developer must:

```
Modify code
↓
Build JAR
↓
Build Docker Image
↓
Push Image
↓
Deploy Again
↓
Test Again
```

Just to change:

```
100 → 50
```

Huge waste.

With ConfigMap

Developer writes:

```java
String dbHost = System.getenv("DB_HOST");

String redisHost = System.getenv("REDIS_HOST");

String logLevel = System.getenv("LOG_LEVEL");

String maxProducts = System.getenv("MAX_PRODUCTS");
```

No values inside code.

Kubernetes ConfigMap

YAML

```yaml
apiVersion: v1
kind: ConfigMap

metadata:
  name: product-config

data:

  DB_HOST: mysql-prod.flipkart.internal

  DB_PORT: "3306"

  REDIS_HOST: redis-prod.flipkart.internal

  REDIS_PORT: "6379"

  LOG_LEVEL: INFO

  MAX_PRODUCTS: "100"

  FEATURE_RECOMMENDATION: "true"
```

Product Pod Reads ConfigMap

YAML

```yaml
apiVersion: apps/v1
kind: Deployment

metadata:
  name: product-service

spec:
  replicas: 3

  template:
    spec:
      containers:

      - name: product

        image: flipkart/product:v1

        envFrom:

        - configMapRef:
            name: product-config
```

What Happens Internally?

Imagine Pod starts.

Kubernetes injects:

```
DB_HOST=mysql-prod.flipkart.internal

DB_PORT=3306

REDIS_HOST=redis-prod.flipkart.internal

LOG_LEVEL=INFO

MAX_PRODUCTS=100
```

inside the container.

Container sees:

```
Bash

env
```

Output:

```
Bash

DB_HOST=mysql-prod.flipkart.internal

REDIS_HOST=redis-prod.flipkart.internal

MAX_PRODUCTS=100
```

Application reads these values.

JIO HOTSTAR EXAMPLE :

Or

HOTSTAR Real-Time Example

Suppose hotstar Streaming Service.

Millions of users watching movies.

Streaming service requires:

```
Video Bitrate

CDN Endpoint

Cache Timeout

Subtitle Language

Streaming Quality
```

ConfigMap

YAML

```yaml
apiVersion: v1
kind: ConfigMap

metadata:
  name: streaming-config

data:

  VIDEO_BITRATE: "4000"

  CACHE_TIMEOUT: "300"

  DEFAULT_LANGUAGE: "English"

  STREAMING_MODE: "HD"
```

During IPL Final

Traffic explodes.

Netflix team decides:

Current:

```yaml
VIDEO_BITRATE=4000
```

Change:

```yaml
VIDEO_BITRATE=2500
```

Benefits:

```
Less bandwidth

Less CDN load

Less buffering

Better user experience
```

No code changes.

INTERVIEW ANSWER:

At companies like Flipkart and Netflix, ConfigMaps are used to externalize application configuration such as database endpoints, cache servers, logging levels, feature flags, API URLs, timeout values, and business rules. This allows operations teams to change application behavior without modifying source code or rebuilding Docker images. During high-traffic events like Big Billion Days or major Netflix releases, ConfigMaps enable quick runtime configuration changes to improve performance, scalability, and operational flexibility

How config map is different then k8s volumes

This is one of the most common Kubernetes interview questions.

Short Answer

ConfigMap	Volume
Stores configuration data	Stores actual files/data
Managed by Kubernetes API	Can come from PV, EBS, EFS, NFS, etc.
Usually KBs of data	Can be GBs or TBs of data
Used for application settings	Used for persistent storage
Not meant for databases	Used by databases and applications
Data can be recreated	Data must be preserved